

BÀI KIỂM TRA NĂNG LỰC KỸ SƯ AI

Mảng: Multi-Agent LLM System & Graph RAG

Mã đề: MAS-LLM-2026-FINAL

Mục lục

1	TỔNG QUAN ĐỀ BÀI	3
1.1	Bối cảnh	3
1.2	Sơ đồ kiến trúc tổng thể	3
1.3	Cấu trúc Phần cứng	3
1.4	Quy tắc nộp bài	4
2	PHẦN A — MULTI-AGENT ROUTER & DATA PIPELINE (35%)	5
2.1	Module A1: Router Agent — Intent Classification (20%)	5
2.1.1	A1.1: Kiến trúc Router	5
2.1.2	A1.2: Data Generation Pipeline cho Router	5
2.1.3	A1.3: Fine-tuning Router (SFT)	6
2.2	Module A2: Multi-Agent System Architecture (20%)	6
2.2.1	A2.1: Agent Framework Design	6
2.2.2	A2.2: Session Management & History	7
2.2.3	A2.3: Request Queue & Concurrency Control	7
3	PHẦN B — GRAPH RAG & LLM SERVING (40%)	8
3.1	Module B1: Graph RAG — Knowledge Base (25%)	8
3.1.1	B1.1: Graph Database Setup	8
3.1.2	B1.2: Vector Index & Hybrid Search	8
3.1.3	B1.3: Data Ingestion Pipeline	9
3.2	Module B2: LLM Serving & Optimization (25%)	9
3.2.1	B2.1: Serving Infrastructure	9
3.2.2	B2.2: Streaming & Token-level Output	10
3.2.3	B2.3: Caching Architecture	10
3.2.4	B2.4: Benchmark & Latency Budget	10
4	PHẦN C — INTELLIGENT CACHE & EDGE (25%)	12
4.1	C1: Edge Deployment — Router trên Orange Pi (5%)	12
4.2	C2: Intelligent Semantic Cache — Intent Extraction & Paraphrase (17%)	12

4.2.1	C2.1: SLM Fine-tuning cho Intent Extraction	12
4.2.2	C2.2: Cache Pipeline & Paraphrase	13
4.3	C3: Production Guardrails (3%)	13
5	BẢNG TỔNG HỢP CHỈ SỐ ĐO LƯỜNG & ĐÁNH GIÁ	14
5.1	Bảng điểm tổng hợp nghiệm thu	14

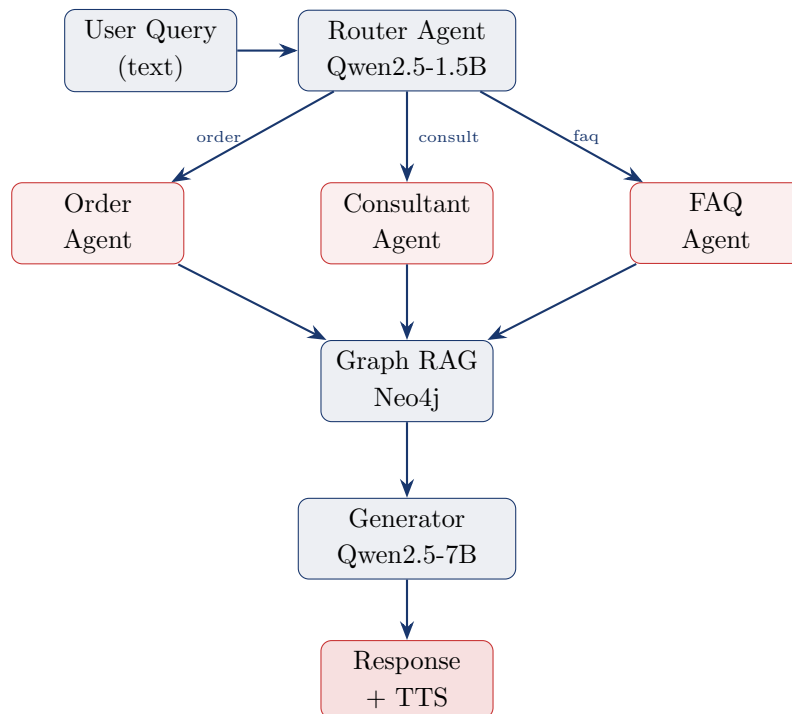
1. TỔNG QUAN ĐỀ BÀI

1.1 Bối cảnh

Công ty FI-AI đang phát triển hệ thống **Robot tư vấn** cho ngành F&B (Highlands Coffee). Hệ thống sử dụng kiến trúc **Multi-Agent** kết hợp **Graph RAG** trên nền tảng hoàn toàn **Local** (không phụ thuộc cloud API) để xử lý 4 luồng chính: *Đặt hàng (Order)*, *Tư vấn (Consultant)*, *Hỏi đáp (FAQ)*, và *Nhận dạng ý định mơ hồ (Ignore/Noise)*.

Ứng viên cần xây dựng toàn bộ pipeline từ **Data Generation**, **Model Fine-tuning**, **Graph Database**, đến **Serving & Optimization** trên phần cứng RTX 3060 12GB VRAM.

1.2 Sơ đồ kiến trúc tổng thể



1.3 Cấu trúc Phần cứng

- **Serving (Sản xuất):** RTX 3060 12GB VRAM — Inference serving có thể tách thành **2 Agent riêng trên 2 máy**: *Router Agent* (machine 1) và *Order/Consultant/FAQ Agent* (machine 2) trong budget VRAM này.
- **Training (Fine-tuning):** Sử dụng tài khoản Trial GPU đám mây (Modal H100, RunPod, Lambda...) cho các tác vụ SFT Router và Edge LLM.
- **Edge (Tùy chọn Bonus):** Orange Pi 6 Max / Chip CIX P1 — Deploy Router dạng GGUF chạy trên CPU. Cần tối ưu để toàn bộ pipeline (Router → Graph RAG → Agent Response → TTS) đạt:

- **TTFA (Time-To-First-Audio) $\leq 3.0s$ là Xuất sắc, $\leq 4.5s$ là Đạt.**

1.4 Quy tắc nộp bài

1. Code phải chạy được end-to-end. Multi-Agent Server serving trên GPU (RTX 3060).
2. Báo cáo kỹ thuật dạng PDF, bao gồm: kiến trúc hệ thống, kết quả benchmark TTFT/Throughput, và phân tích thất bại.
3. Demo video $\geq 60s$ — thể hiện luồng hội thoại đa lượt (multi-turn conversation) qua ít nhất 3 agents khác nhau.
4. Repository Git với commit history rõ ràng.

2. PHẦN A — MULTI-AGENT ROUTER & DATA PIPELINE (35%)

2.1 Module A1: Router Agent — Intent Classification (20%)

2.1.1 A1.1: Kiến trúc Router

Bối cảnh: Router là “não” phân luồng của hệ thống. Nó nhận câu hỏi từ người dùng và phân loại (classify) thành đúng 1 trong 4 intents:

Intent	Label	Ví dụ
order	0	“Cho anh 1 ly cà phê sữa”, “Tính tiền”, “Gọi thêm bánh”
consultant	1	“Có gì ngon không?”, “Gợi ý cho tôi món nào?”
faq	2	“Wifi tên gì?”, “Mấy giờ đóng cửa?”
ignore	3	“Ừm...”, “Hello”, “haha”, tiếng ồn

Bảng 1: Bảng phân loại 4 Intents

Yêu cầu:

- Sử dụng **Small Language Model** (0.5B – 1.5B params) làm Router, ví dụ: Qwen2.5-1.5B-Instruct hoặc Qwen3-0.6B.
- Router phải trả kết quả dạng **JSON chuẩn**: {"action": "<intent>"}
- Latency mục tiêu:** Thời gian phân loại (từ nhận text → trả intent) $\leq 200\text{ms}$ trên RTX 3060, $\leq 100\text{ms}$ là Xuất sắc.
- Hỗ trợ tối thiểu **2 ngôn ngữ** (Việt, Anh). Hỗ trợ thêm ngôn ngữ thứ 3+ là điểm cộng.

2.1.2 A1.2: Data Generation Pipeline cho Router

Yêu cầu:

- Thiết kế pipeline tự động sinh dữ liệu huấn luyện (Synthetic Data Generation) cho Router.
- Tối thiểu **4000 samples**, phân bổ cân bằng đều cho 4 intents (~25% mỗi loại).
- Bao gồm **Hard samples** (~5–20%): các câu mơ hồ, lưỡng lự giữa 2 intents (ví dụ: “Có gì ngon rẻ không?” — order hay consultant?).
- Pipeline phải có: async API calls, deduplication, checkpoint/resume, và progress tracking.
- Dữ liệu output dạng JSON chứa: text, label (0-3), intent, is_noise, language.

2.1.3 A1.3: Fine-tuning Router (SFT)

Yêu cầu:

- Fine-tune model router bằng SFT trên bộ dữ liệu đã sinh.
- Training trên GPU đám mây (H100 Trial), xuất model thành định dạng phục vụ serving:
 - **AWQ/GPTQ Quantization** cho serving SGLang/vLLM trên RTX 3060.
 - **GGUF Quantization** (Q4_K_M hoặc IQ4_XS) cho Edge deployment (Bonus).
- **Accuracy mục tiêu:** $\geq 92\%$ trên test set cân bằng (4 classes).
- Phải benchmark: **Confusion matrix**, **per-class F1-score**, phân tích chi tiết lỗi phân loại.

Tiêu chí nghiệm thu A1:

- Router accuracy $\geq 92\%$ trên test set cân bằng.
- **Chỉ số mục tiêu — Latency phân loại:** $\leq 200\text{ms}$ là Đạt, $\leq 100\text{ms}$ là Xuất sắc (RTX 3060).
- JSON output chuẩn 100%, không lỗi parse.
- Hard samples (ambiguous) accuracy $\geq 75\%$.

2.2 Module A2: Multi-Agent System Architecture (20%)

2.2.1 A2.1: Agent Framework Design

Yêu cầu:

- Thiết kế hệ thống **Multi-Agent** gồm 1 **Router Agent** (cấp quản lý) phân luồng công việc cho ít nhất 3 **Specialized Agents**:
 - **Order Agent:** Xử lý đặt hàng, thêm/bớt món, tính tiền. Kết nối với dữ liệu Menu trong Graph DB.
 - **Consultant Agent:** Tư vấn, gợi ý món dựa trên ngữ cảnh (khẩu vị, ngân sách, thời tiết). Sử dụng RAG từ Menu + FAQ.
 - **FAQ Agent:** Hỏi đáp chung (wifi, giờ mở cửa, chính sách). Pipeline: Hybrid RAG (Dual-Domain Search $\rightarrow$ Graph Expansion $\rightarrow$ Late Reranking).
- Mỗi Agent phải có **System Prompt riêng biệt** phù hợp với vai trò.
- Hỗ trợ đa ngôn ngữ (ít nhất Việt + Anh): Agent tự phản hồi theo ngôn ngữ đầu vào.

2.2.2 A2.2: Session Management & History

Yêu cầu:

- Thiết kế **SessionStore** quản lý lịch sử hội thoại theo `session_id`.
- Lưu trữ tối thiểu **5 câu hỏi gần nhất** (history window) để duy trì ngữ cảnh multi-turn.
- **Auto-summarization**: Khi số lượng lịch sử vượt quá ngưỡng token cho phép ($\sim 70\%$ context window), tự động gọi LLM tóm tắt phần cũ thành 1 đoạn ngắn.
- **Session TTL**: Tự động xóa session không hoạt động sau > 30 phút.
- **Background Cleanup**: Chạy task dọn rác định kỳ, giải phóng bộ nhớ.

2.2.3 A2.3: Request Queue & Concurrency Control

Yêu cầu:

- Thiết kế **RequestQueue** sử dụng `asyncio.Semaphore` để giới hạn số lượng request LLM đồng thời (tránh OOM trên RTX 3060).
- Cơ chế FIFO với timeout: Request chờ quá lâu ($> 60s$) phải trả về lỗi graceful.
- **Retry with Exponential Backoff**: Mỗi API call (Router, Generator, Embedding) phải có retry tối thiểu 3 lần với delay tăng dần.

Tiêu chí nghiệm thu A2:

- Multi-Agent xử lý đúng luồng: Order $\rightarrow$ Menu lookup, FAQ $\rightarrow$ RAG search, Consultant $\rightarrow$ Hybrid search.
- Session history hoạt động đúng qua ≥ 5 turns liên tục.
- Không xảy ra OOM khi 3+ requests đồng thời.
- Zero deadlock, zero memory leak trong 30 phút chạy liên tục.

3. PHẦN B — GRAPH RAG & LLM SERVING (40%)

3.1 Module B1: Graph RAG — Knowledge Base (25%)

3.1.1 B1.1: Graph Database Setup

Yêu cầu:

- Triển khai **Graph Database** sử dụng **Neo4j** hoặc **FalkorDB** để lưu trữ toàn bộ knowledge base.
- Thiết kế schema Graph gồm tối thiểu các node types:
 - **MenuItem**: Tên món, giá, size, category, ingredients, mô tả.
 - **Chunk**: Đoạn văn bản đã chunked (FAQ, tài liệu nội bộ).
 - **Entity**: Thực thể được trích xuất từ văn bản (tên món, địa điểm, chính sách...).
- Relationships: NEXT (giữa các Chunk liên kề), MENTIONS (Chunk $\rightarrow$ Entity), BELONGS_TO (MenuItem $\rightarrow$ Category).

3.1.2 B1.2: Vector Index & Hybrid Search

Yêu cầu:

- Tạo **Vector Index** trên Graph DB cho phép tìm kiếm ngữ nghĩa (semantic search):
 - Index menu_embedding: Vector search trên MenuItem.
 - Index chunk_embedding: Vector search trên Chunk (FAQ + Document).
- **Embedding Model**: Sử dụng mô hình embedding đa ngôn ngữ, ví dụ: Qwen3-Embedding-0.6B hoặc tương đương. Serve qua **TEI** (Text Embeddings Inference) hoặc tự build API.
- **Reranker**: Tích hợp mô hình Reranker (BGE Reranker v2 hoặc tương đương) để sắp xếp lại kết quả sau vector search.
- **Hybrid Search Pipeline**:
 1. Dual-Domain Search: Song song tìm kiếm trên cả FAQ và Document.
 2. Graph Expansion: Mở rộng kết quả bằng cách duyệt Chunk liên kề (PREV/NEXT) và Entity liên quan (MENTIONS).
 3. Late Reranking: Rerank toàn bộ kết quả mở rộng, lọc theo ngưỡng relevance (≥ 0.7).

3.1.3 B1.3: Data Ingestion Pipeline

Yêu cầu:

- Xây dựng pipeline nạp dữ liệu (Ingestion) từ nhiều nguồn:
 - **Menu CSV:** Parse, tạo embedding, lưu vào Graph.
 - **FAQ CSV:** Tách cặp Q&A, embedding, lưu kèm metadata.
 - **PDF/DOCX:** Semantic Chunking (gradient breakpoint), Entity Extraction tự động bằng LLM, tạo relationships.
- **Entity Deduplication:** Fuzzy matching (Jaccard similarity ≥ 0.85) để gộp các entity trùng lặp.
- **Watch Mode:** Tự động detect file mới trong thư mục và ingest realtime.

Tiêu chí nghiệm thu B1:

- Graph DB chứa ≥ 100 MenuItem, ≥ 150 FAQ chunks, ≥ 50 Document chunks.
- Vector search trả về kết quả relevant (top-5 precision $\geq 80\%$) cho 20 câu hỏi test.
- Graph Expansion tăng recall $\geq 15\%$ so với vector search thuần.
- **Zero hallucination:** LLM không sinh thông tin ngoài context được cung cấp.

3.2 Module B2: LLM Serving & Optimization (25%)

3.2.1 B2.1: Serving Infrastructure

Yêu cầu:

- Triển khai serving LLM sử dụng **SGLang** hoặc **vLLM** (ứng viên tự chọn và biện luận trade-off).
- Kiến trúc **Dual-model** trên cùng 1 GPU (12GB VRAM):
 - **Router model** (0.5B – 1.5B params, AWQ): VRAM budget $\sim 1.1\text{GB}$ (10% VRAM).
 - **Generator model** (7B params, AWQ): VRAM budget $\sim 4.8\text{GB}$ (78% VRAM còn lại).
- Cấu hình **SGLang tuning** cần tối ưu:
 - `mem-fraction-static`: Phân bổ VRAM cho KV Cache.
 - `context-length`: Router ~ 512 , Generator ~ 4096 .
 - `schedule-policy`: `lpm` (Longest Prefix Match) để tận dụng prefix caching.
 - `chunked-prefill-size`, `max-running-requests`: Cân bằng throughput vs latency.
- Toàn bộ hệ thống phải container hóa bằng **Docker Compose**, khởi động theo đúng thứ tự dependency.

3.2.2 B2.2: Streaming & Token-level Output

Yêu cầu:

- Triển khai **True Token Streaming** via SSE (Server-Sent Events): LLM Generator phải stream token-by-token qua API.
- Xử lý đúng [DONE] signal trong SSE stream (không được parse [DONE] như JSON).
- **Clause-level Streaming cho TTS:** Nếu tích hợp TTS downstream, phải tách text tại dấu câu (., ? ! ;) và gửi TTS từng mệnh đề (clause) thay vì đợi toàn bộ response.

3.2.3 B2.3: Caching Architecture

Bối cảnh: Trong ngành F&B, rất nhiều câu hỏi lặp đi lặp lại (“Wifi gì?”, “Có gì ngon?”, “Giờ đóng cửa?”). Thiết kế cache thông minh là lợi thế cạnh tranh lớn.

Yêu cầu:

- Thiết kế hệ thống cache **đa tầng** (Multi-layer Caching):
 - **Layer 1 — Model Cache (Disk):** Cache model weights tránh re-download.
 - **Layer 2 — Graph Cache (Persistent DB):** Dữ liệu RAG đã được tiền xử lý và embedding sẵn trong Neo4j.
 - **Layer 3 — KV Cache (VRAM):** SGLang/vLLM KV Cache cho prefix-sharing giữa các request.
 - **Layer 4 — Session Cache (In-Memory):** Lưu history hội thoại theo session_id.
- **Semantic Cache (Điểm cộng lớn):** Cache kết quả câu hỏi thường xuyên lặp lại. Dùng embedding similarity (≥ 0.95) để phát hiện câu hỏi tương tự $\rightarrow$ trả kết quả cached ngay lập tức mà không cần gọi LLM.
- Phân tích rõ: mỗi layer cache chiếm bao nhiêu tài nguyên (RAM/VRAM/Disk) và TTL bao lâu.

3.2.4 B2.4: Benchmark & Latency Budget

Chỉ số đo lường:

- **TTFT (Time-To-First-Token):** Thời gian từ lúc gửi request đến khi nhận được token đầu tiên.
 - $\leq 0.2s$ là **Đạt**, $\leq 0.07s$ là **Xuất sắc** (trên RTX 3060 — cần ứng viên benchmark thực tế và báo cáo).
- **Throughput:** Số tokens/giây khi serving đồng thời nhiều requests.
- **Total Pipeline Latency:** Từ lúc user gửi query $\rightarrow$ nhận được response hoàn chỉnh.
 - Mục tiêu: $\leq 1.5s$ (Đạt), $\leq 0.8s$ (Xuất sắc).

Tiêu chí nghiệm thu B2:

- Dual-model serving ổn định trên 12GB VRAM, không OOM.
- TTFT $\leq 0.2s$ (Generator, RTX 3060).
- Streaming SSE hoạt động đúng, không mất token.
- Cache hệ thống hoạt động: lần query thứ 2 cho câu hỏi tương tự phải nhanh hơn đáng kể.
- **Zero hallucination:** LLM trả lời hoàn toàn dựa trên context từ RAG, không bịa thông tin (đặc biệt về giá, menu, chính sách).

4. PHẦN C — INTELLIGENT CACHE & EDGE (25%)

4.1 C1: Edge Deployment — Router trên Orange Pi (5%)

Bối cảnh: Trong sản xuất thực tế, Router cần chạy ngay trên thiết bị Edge (Orange Pi 6 Max / Chip CIX P1) để giảm latency mạng.

Yêu cầu:

- Xuất model Router sang định dạng **GGUF** (llama.cpp) hoặc **MNN** (Alibaba Mobile Neural Network).
- Benchmark so sánh các quant level: Q4_0, Q4_K_M, Q4_K_S, IQ4_XS trên thiết bị ARM.
- Báo cáo: Accuracy vs Latency vs Memory trade-off trên từng quant level.
- **Mục tiêu Edge:** Router classify $\leq 500\text{ms}$ trên CPU ARM (CIX P1).

4.2 C2: Intelligent Semantic Cache — Intent Extraction & Paraphrase (17%)

Bối cảnh: Đây là module **cực kỳ quan trọng** quyết định tốc độ phản hồi của toàn bộ hệ thống. Trong F&B, rất nhiều câu hỏi lặp đi lặp lại nhưng ngữ cảnh khác nhau. Ứng viên cần bắt buộc thiết kế cache thông minh có khả năng **tách yêu cầu chính** khỏi ngữ cảnh để query chính xác.

4.2.1 C2.1: SLM Fine-tuning cho Intent Extraction

Yêu cầu:

- Bắt buộc **fine-tune một Small Language Model < 3B** (ví dụ: Qwen2.5-0.5B, Qwen3-0.6B) để thực hiện tác vụ **Intent Extraction** — phân tách câu nói của khách thành 3 thành phần cấu trúc:
 - **{Chủ ngữ}**: Ai đang nói / yêu cầu.
 - **{Hành động / Yêu cầu}**: Động từ chính, nội dung cốt lõi.
 - **{Ngữ cảnh / Thời gian / Bối cảnh}**: Các yếu tố phụ trợ (thời gian, số lượng người, dịp đặc biệt...).
- **Ví dụ minh họa:**

Input: “Cho anh đặt bàn tiệc sinh nhật vào ngày mai lúc 7h em nhé”

Output:

- Chủ ngữ: “anh” (khách hàng)
- Hành động: “đặt bàn tiệc sinh nhật” → **cache key**

– Ngữ cảnh: “ngày mai lúc 7h” → gửi kèm sang Agent

- Phần **Hành động/Yêu cầu** được dùng làm **cache key** để query Semantic Cache DB. Loại bỏ các yếu tố thời gian, ngữ cảnh, bối cảnh ra khỏi cache key để tăng **cache hit rate**.

4.2.2 C2.2: Cache Pipeline & Paraphrase

Luồng xử lý:

1. **Extract:** SLM tách câu thành {Hành động} + {Ngữ cảnh}.
2. **Query Cache:** Dùng embedding similarity (≥ 0.92) trên phần {Hành động} để tìm kiếm trong Cache DB.
3. **Cache Hit** → **Paraphrase trực tiếp:** Lấy response template từ cache, kết hợp với phần {Ngữ cảnh} để sinh câu trả lời hoàn chỉnh — dạng paraphrase mượt mà, thân thiện, lịch sự, chuyên nghiệp trong môi trường F&B. Xử lý ngay tại **RAM** mà không cần gọi Agent.
4. **Cache Miss** → **Agent:** Gửi toàn bộ (hành động + ngữ cảnh) sang Agent tương ứng (Order/Consultant/FAQ) để xử lý, đồng thời lưu kết quả vào cache.

Tiêu chí nghiệm thu C2:

- **Cache hit rate** $\geq 60\%$ trên bộ 500 câu hỏi F&B thực tế (có biến thể ngữ cảnh).
- SLM Intent Extraction accuracy $\geq 90\%$ trên test set.
- Câu paraphrase output phải tự nhiên, dễ nghe, gần gũi — phù hợp phát qua TTS.
- Cơ chế **cache invalidation** khi menu/FAQ thay đổi.
- Latency cache-hit path: $\leq 100\text{ms}$ (không qua LLM Generator).

4.3 C3: Production Guardrails (3%)

- TTS Preprocessing: Xử lý giá tiền Việt Nam (“49.000đ” → “49k”), đơn vị size (“M” → “mờ”), VAT → “thuế”.
- Rate limiting cho API endpoints.
- Graceful degradation: Router fallback khi Generator quá tải.
- Health check endpoints cho tất cả services.

5. BẢNG TỔNG HỢP CHỈ SỐ ĐO LƯỜNG & ĐÁNH GIÁ

5.1 Bảng điểm tổng hợp nghiệm thu

Mã	Hạng mục	Trọng số	Đạt	Xuất sắc
PHẦN A — Multi-Agent & Router (35%)				
A1.1	Router Accuracy (4 classes)	7%	☐	☐
A1.2	Data Generation Pipeline	5%	☐	☐
A1.3	Router SFT & Quantization	6%	☐	☐
A2.1	Multi-Agent Framework	7%	☐	☐
A2.2	Session & History Management	5%	☐	☐
A2.3	Concurrency & Queue Control	5%	☐	☐
PHẦN B — Graph RAG & LLM Serving (40%)				
B1.1	Graph Database Schema	4%	☐	☐
B1.2	Vector Index & Hybrid Search	8%	☐	☐
B1.3	Data Ingestion Pipeline	4%	☐	☐
B1	Zero Hallucination (RAG)	5%	☐	☐
B2.1	LLM Serving (SGLang/vLLM)	7%	☐	☐
B2.2	Streaming & SSE	3%	☐	☐
B2.3	Caching Architecture (Multi-layer)	5%	☐	☐
B2.4	TTFT & Latency Benchmark	4%	☐	☐
PHẦN C — Intelligent Cache & Edge (25%)				
C1	Edge Deployment (GGUF/MNN)	5%	☐	☐
C2.1	SLM Intent Extraction (<3B)	7%	☐	☐
C2.2	Cache Pipeline & Paraphrase	10%	☐	☐
C3	Production Guardrails	3%	☐	☐
TỔNG		100%		

Bảng 2: Bảng điểm tổng hợp nghiệm thu — Multi-Agent LLM System

Tổng điểm	Xếp loại	Quyết định
≥ 92%	Xuất sắc	Nhận vị trí AI Engineer; xem xét Lead
75 – 91%	Đạt	Nhận vị trí AI Engineer
66 – 74%	Cần cải thiện	Xem xét thực tập
< 66%	Không đạt	Từ chối

Bảng 3: Bảng phân loại kết quả nghiệm thu

Thứ tự ưu tiên chấm điểm (từ cao → thấp):

1. **Độ chính xác & Zero Hallucination:** Model trả lời đúng, không bịa — đây là tiêu chí tối thượng.
2. **Kiến trúc Multi-Agent & Graph RAG:** Hệ thống phân luồng đúng, RAG truy xuất chính xác.
3. **LLM Serving & Optimization:** TTFT thấp, VRAM budget gọn, serving ổn định.
4. **Caching & Production Readiness:** Cache đa tầng, session management, guardrails.